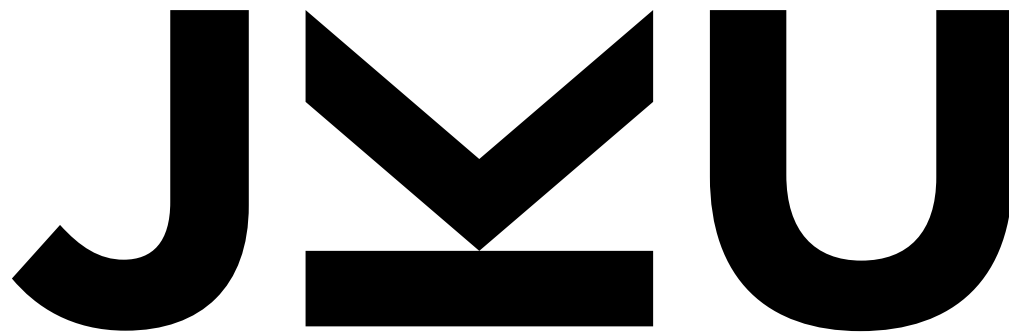




**JOHANNES KEPLER
UNIVERSITY LINZ**

Deep Learning and Neural Networks II: 0. Recap on RNNs



Günter Klambauer
LIT AI Lab & Institute for Machine Learning

JOHANNES KEPLER
UNIVERSITY LINZ
Altenberger Strasse 69
4040 Linz, Austria
jku.at

This material, no matter whether in printed or electronic form, may be used for personal and non-commercial educational use only. Any reproduction of this material, no matter whether as a whole or in parts, no matter whether in printed or in electronic form, requires explicit prior acceptance of the authors.

Recap: Notation for RNNs

- Input objects (features) at time point t : $\mathbf{x}_t$
- Input sequence: $(\mathbf{x}_t)_{t=1}^T$
 - Abbreviation: $\mathbf{x}_{(1:T)}$
- Weight matrices in RNNs: $\mathbf{W}, \mathbf{R}, \mathbf{V}$
- Weight matrix in a neural net in layer l : $\mathbf{W}^{(l)}$
- All weights in a neural net (vector): $\mathbf{w}$
- Model function: $g(\mathbf{x}; \mathbf{w})$
- Predictions at time point t : $\hat{\mathbf{y}}_t$

Time series data as input

- Input objects at time point t

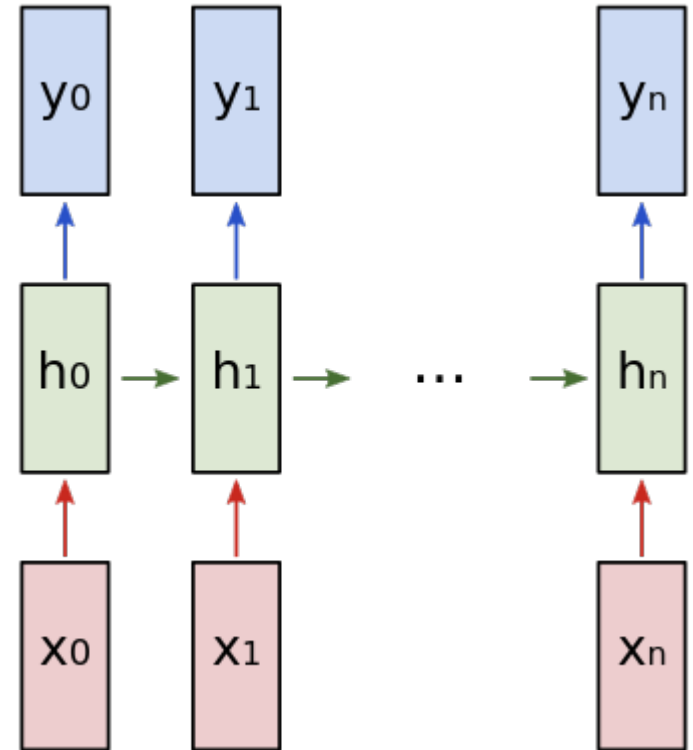
$$\mathbf{x}(t)$$

- Input sequence:

$$(\mathbf{x}(t))_{t=1}^T$$

- Abbreviation:

$$\mathbf{x}(1:T)$$



Recurrent neural networks (RNNs)

- Map an input sequence to an output sequence:

$$\hat{y}(t) = g(a(0), x(1), \dots, x(t); w)$$

or:

$$\hat{y}(t) = g(a(0), x(1 : t); w)$$

- Quite common is to use the following:

$$\hat{y}(t) = g(h(0), x(1 : t); w)$$

Recap: Recurrent neural net (RNN)

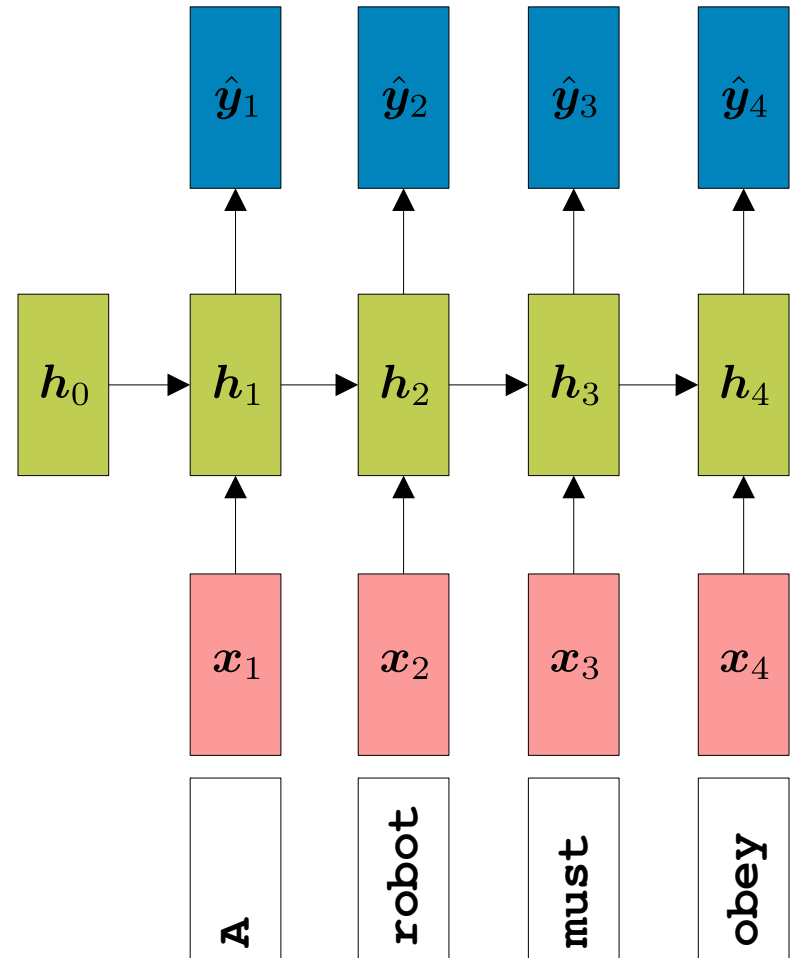
- Recurrent neural network:

$$h_t = \text{RNN}(x_t, h_{t-1}; w)$$

- Fully recurrent network

$$h_t = f(Wx_t + Rh_{t-1})$$

$$\hat{y}_t = \phi(Vh_t)$$



Recap: Recurrent neural net (RNN)

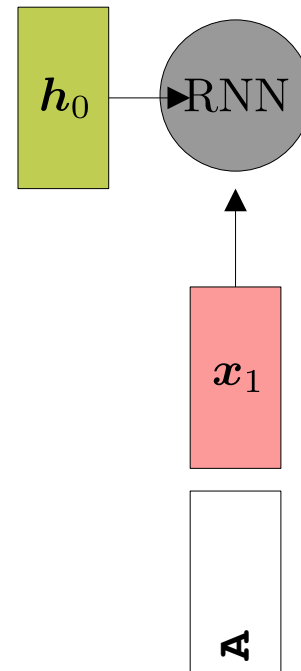
- Recurrent neural network:

$$h_t = \text{RNN}(x_t, h_{t-1}; w)$$

- Fully recurrent network

$$h_t = f(Wx_t + Rh_{t-1})$$

$$\hat{y}_t = \phi(Vh_t)$$



Recap: Recurrent neural net (RNN)

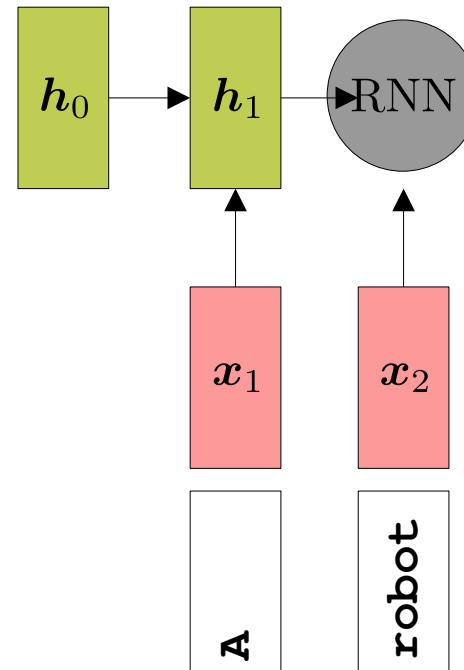
- Recurrent neural network:

$$h_t = \text{RNN}(x_t, h_{t-1}; w)$$

- Fully recurrent network

$$h_t = f(Wx_t + Rh_{t-1})$$

$$\hat{y}_t = \phi(Vh_t)$$



Recap: Recurrent neural net (RNN)

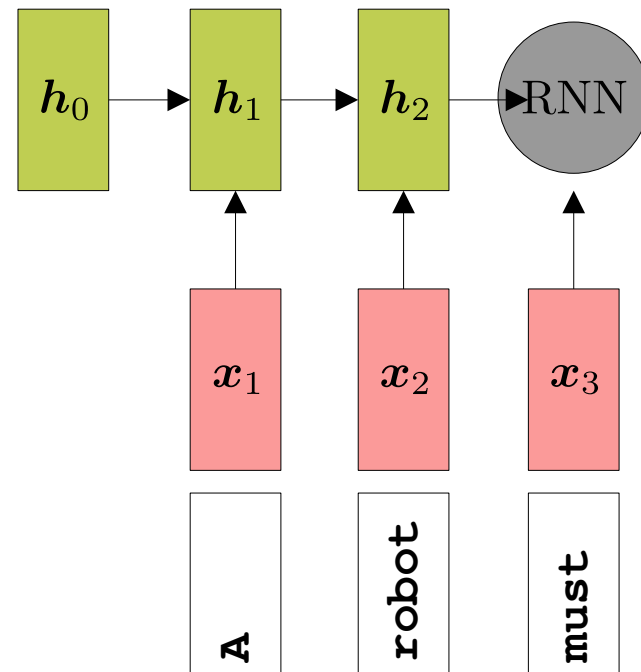
- Recurrent neural network:

$$h_t = \text{RNN}(x_t, h_{t-1}; w)$$

- Fully recurrent network

$$h_t = f(Wx_t + Rh_{t-1})$$

$$\hat{y}_t = \phi(Vh_t)$$



Recap: Recurrent neural net (RNN)

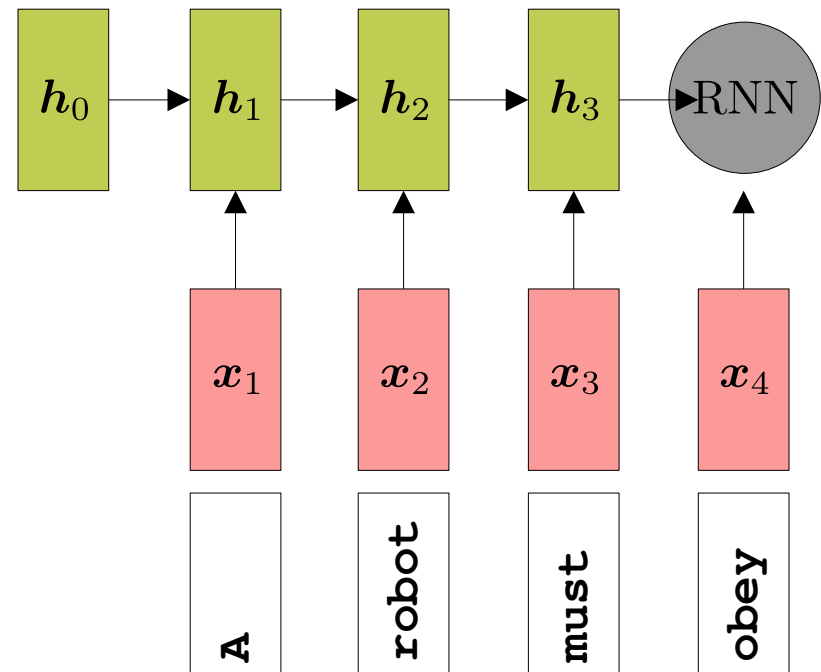
- Recurrent neural network:

$$h_t = \text{RNN}(x_t, h_{t-1}; w)$$

- Fully recurrent network

$$h_t = f(Wx_t + Rh_{t-1})$$

$$\hat{y}_t = \phi(Vh_t)$$



Recap: Recurrent neural net (RNN)

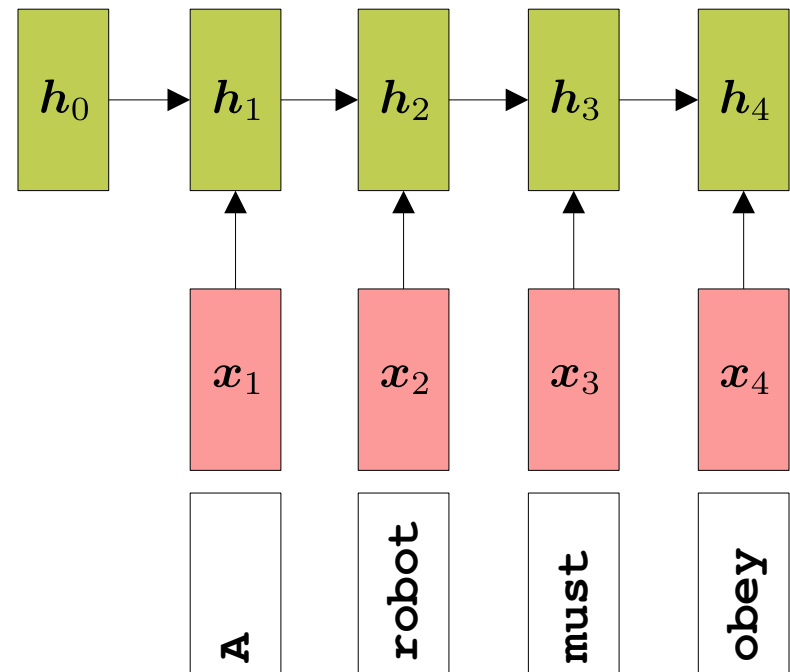
- Recurrent neural network:

$$h_t = \text{RNN}(x_t, h_{t-1}; w)$$

- Fully recurrent network

$$h_t = f(Wx_t + Rh_{t-1})$$

$$\hat{y}_t = \phi(Vh_t)$$



Recap: Recurrent neural net (RNN)

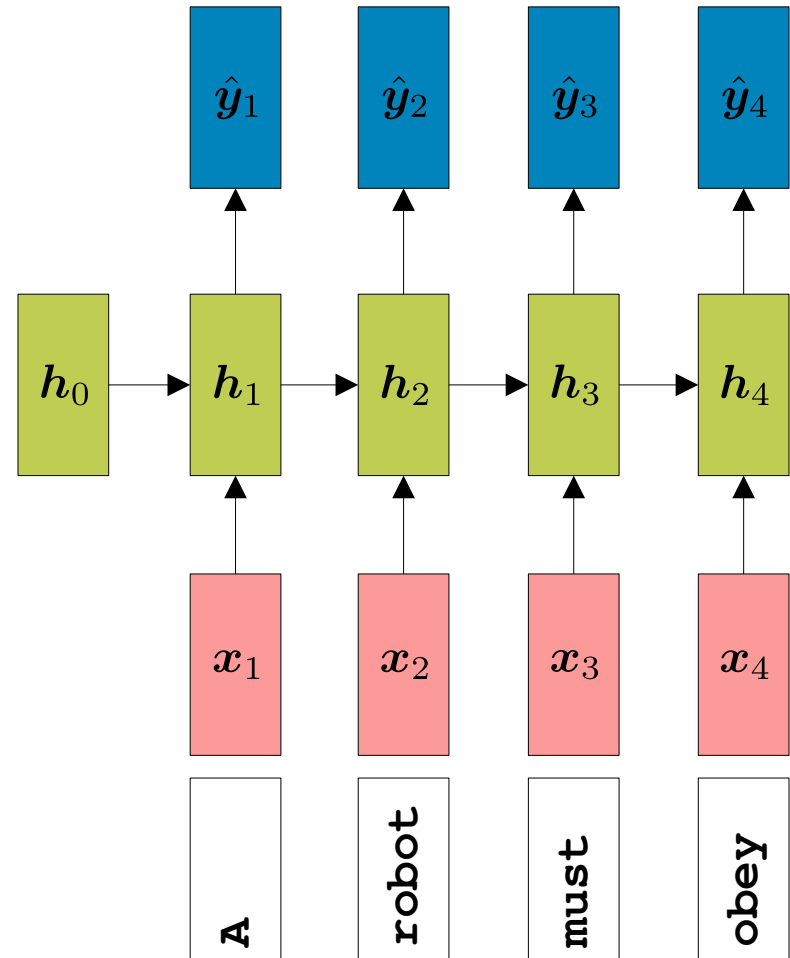
- Recurrent neural network:

$$h_t = \text{RNN}(x_t, h_{t-1}; w)$$

- Fully recurrent network

$$h_t = f(Wx_t + Rh_{t-1})$$

$$\hat{y}_t = \phi(Vh_t)$$



Recap: Recurrent neural net (RNN)

- Recurrent neural network:

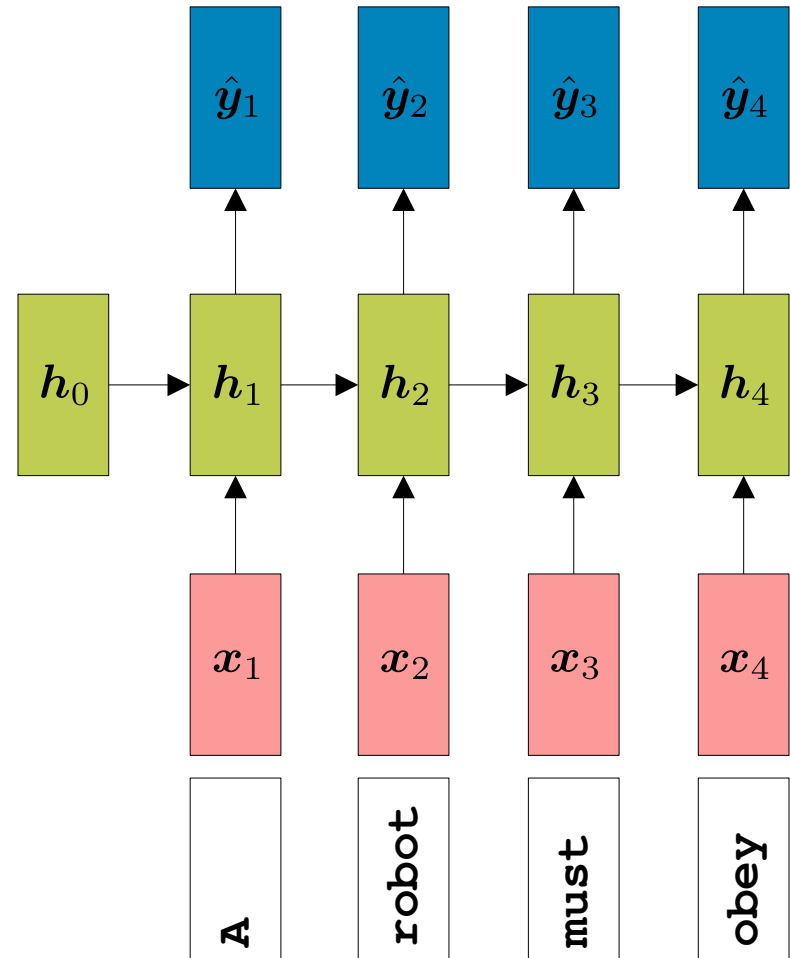
$$h_t = \text{RNN}(x_t, h_{t-1}; w)$$

- Fully recurrent network

$$h_t = f(Wx_t + Rh_{t-1})$$

$$\hat{y}_t = \phi(Vh_t)$$

- Note that the last layer could be processed in **parallel**



Transformer: self-attention

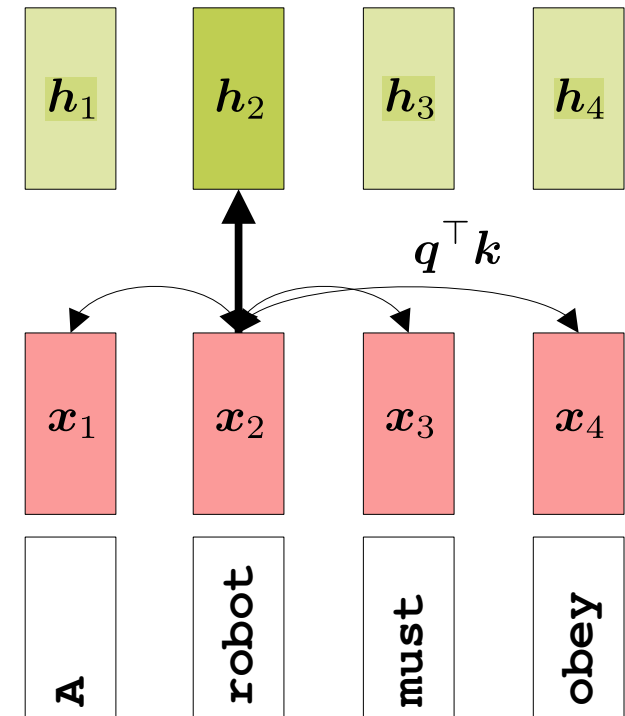
- Uses a set of inputs (rather than a single input)

$$\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_t, \dots, \mathbf{x}_T\}$$

- Note that $\mathbf{X} \in \mathbb{R}^{D \times T}$ is not the training data matrix, but the time series of inputs. It contains positions encoded as features (position encoding). Otherwise: permutation-equivariant
- The forward pass of a transformer is given by

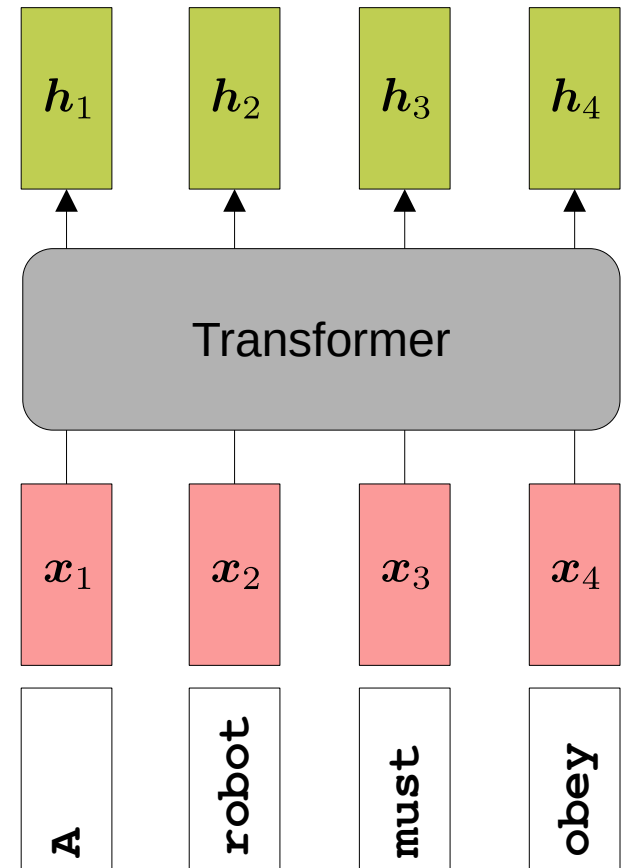
Transformer: self-attention

- Attention mechanism
 - Dot product between
 - Keys
 - Queries
 - All time steps (tokens) can be processed in parallel
 - Softmax for sparseness
 - Quadratic in time steps



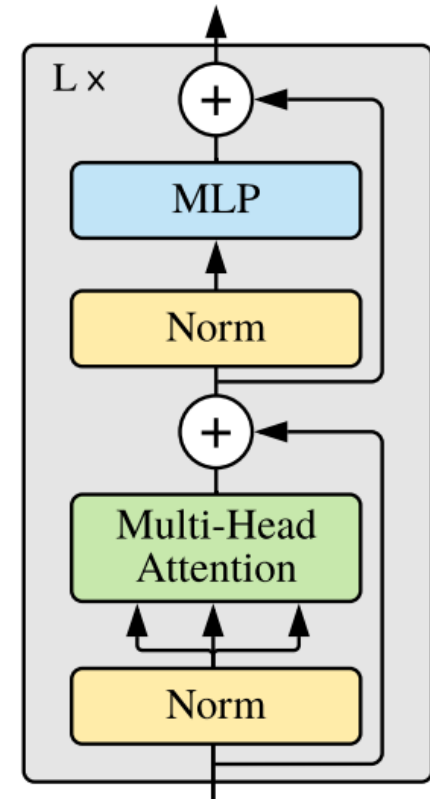
Transformer: self-attention

- Attention mechanism
 - Dot product between
 - Keys
 - Queries
 - All time steps (tokens) can be processed in parallel
 - Softmax for sparseness
 - Quadratic in time steps

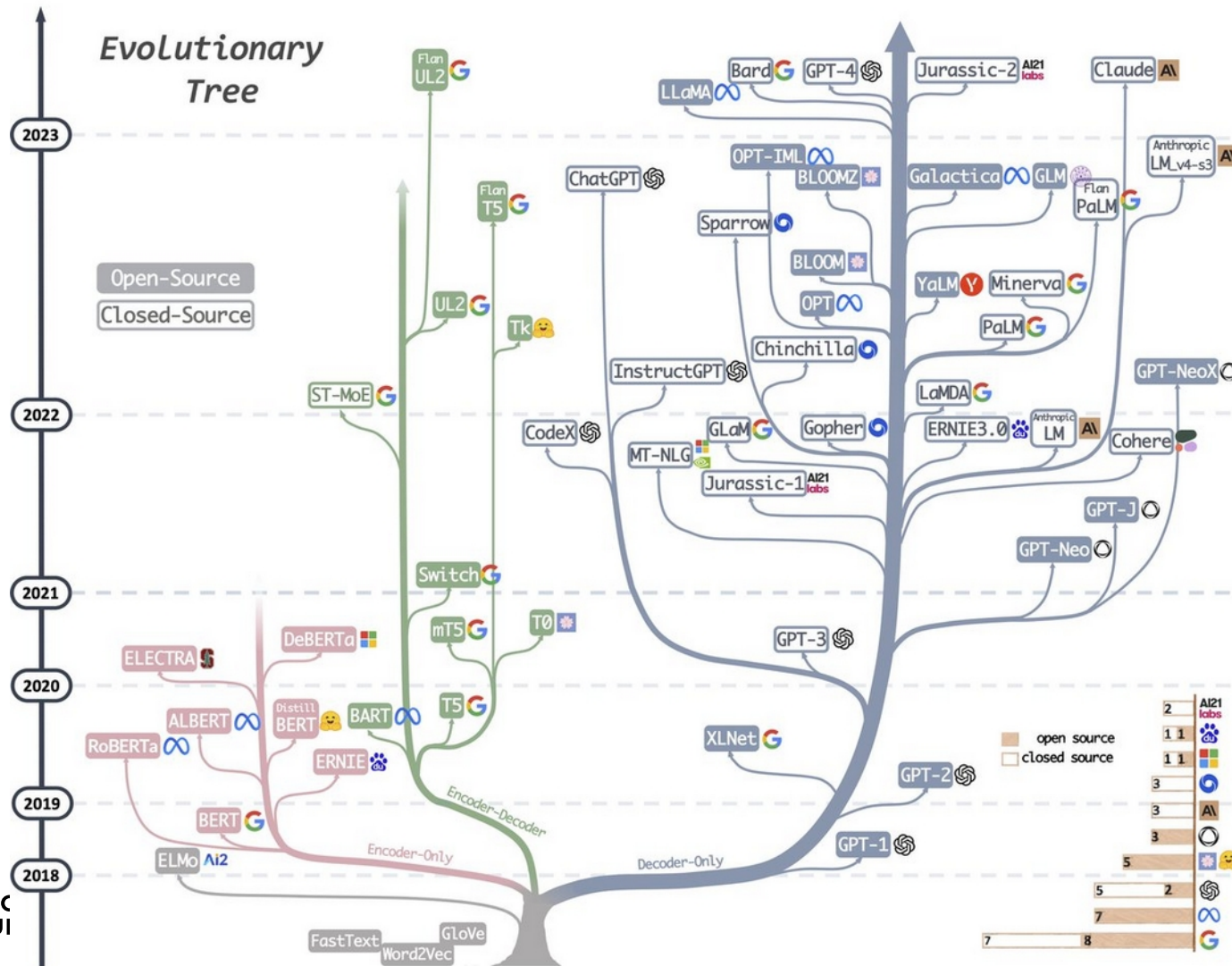


Transformer: layer and architecture

- The **transformer layer** consists of
 - (optional) positional encoding
 - Layer normalization
 - Self-attention mechanism
 - Skip connection
 - Layer normalization (again)
 - Multi-layer perceptron
 - Skip connection (again)

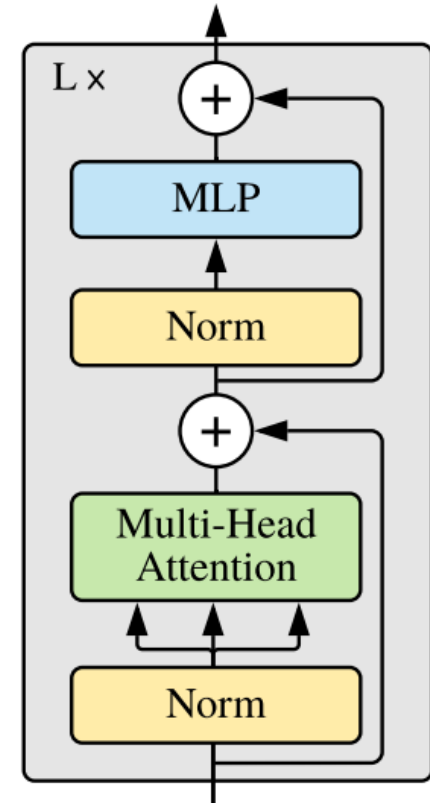


Evolution of Large Language Models (LLMs)



Transformers and LLMs

- **Architecture**
 - Explanation of multi-head attention
 - Positional encoding
 - Layer normalization
- **Pre-training** via masking (BERT)
 - Encoder-type transformers
- **Generative pre-training** (GPT)
 - Decoder-type transformers
- **Scaling laws**

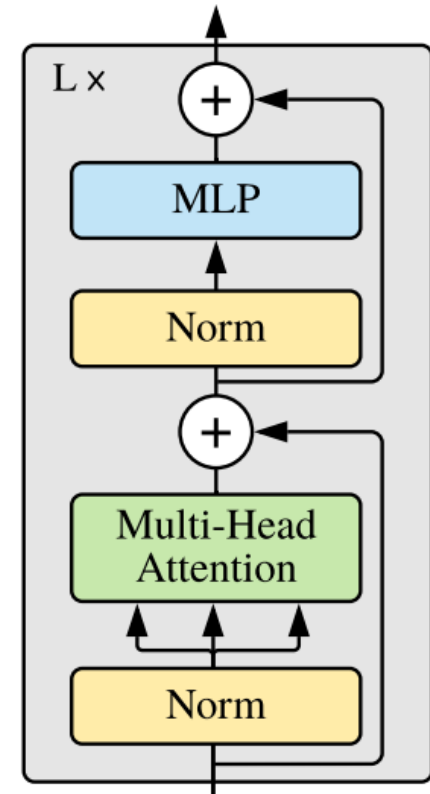


Summary

- Transformers operate on sets of input vectors (tokens)

$$\mathbf{H}^{(l)} = \text{Transformer}(\mathbf{H}^{(l-1)}; \mathbf{W}_K, \mathbf{W}_Q, \mathbf{W}_V)$$

- Main operation is **self-attention**
- High-parallelism
- Permutation-equivariant**
- Basis of large language models (LLMs)
 - BERT, GPT
- Special pre-training strategies necessary (SOON!)



Transformer self-attention: revisited

- Uses a set of inputs (rather than a single input);

$$\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_t, \dots, \mathbf{x}_T\}$$

Note that $\mathbf{X} \in \mathbb{R}^{D \times T}$ is not the training data matrix, but the time series of inputs. Input contains positions encoded as features (position encoding). Otherwise: permutation-equivariant

- The forward pass of a transformer is given by

$$\mathbf{H}^0 = \mathbf{X} \tag{1}$$

$$\mathbf{K}^{(l)} = \mathbf{W}_K^{(l)} \mathbf{H}^{(l-1)} \tag{2}$$

$$\mathbf{Q}^{(l)} = \mathbf{W}_Q^{(l)} \mathbf{H}^{(l-1)} \tag{3}$$

$$\mathbf{V}^{(l)} = \mathbf{W}_V^{(l)} \mathbf{H}^{(l-1)} \tag{4}$$

$$\mathbf{H}^{(l)} = \underbrace{\text{softmax} \left(1/\sqrt{d_k} \mathbf{Q}^{(l)T} \mathbf{K}^{(l)} \right)}_{:=\mathbf{A}^{(l)}} \mathbf{V}^{(l)}, \tag{5}$$

Now, in a bit more detail...

1.1 Jordan networks

- Forward pass:

$$\mathbf{s}(t) = \mathbf{W}^\top \mathbf{x}(t) + \mathbf{R}^\top \hat{\mathbf{y}}(t-1)$$

$$\mathbf{a}(t) = f(\mathbf{s}(t))$$

$$\hat{\mathbf{y}}(t) = \varphi(\mathbf{V}^\top \mathbf{a}(t)),$$

- Parameters:

$$\mathbf{W} \in \mathbb{R}^{D \times I}, \mathbf{R} \in \mathbb{R}^{K \times I}, \mathbf{V} \in \mathbb{R}^{I \times K}$$

1.3 Fully recurrent network

- Forward pass:

$$\mathbf{s}(t) = \mathbf{W}^\top \mathbf{x}(t) + \mathbf{R}^\top \mathbf{a}(t-1)$$

$$\mathbf{a}(t) = f(\mathbf{s}(t))$$

$$\hat{\mathbf{y}}(t) = \varphi(\mathbf{V}^\top \mathbf{a}(t)),$$

- Parameters:

$$\mathbf{W} \in \mathbb{R}^{D \times I}, \mathbf{R} \in \mathbb{R}^{I \times I}, \mathbf{V} \in \mathbb{R}^{I \times K}$$

2.1 Learning by empirical risk minimization (ERM)

- In analogy to learning FCNs and CNNs, recurrent neural networks minimize a differentiable objective:

$$\begin{aligned} R_{\text{emp}} \left(\left\{ \mathbf{y}(1:T)^{(n)}, \hat{\mathbf{y}}(1:T)^{(n)} \right\}_{n=1}^N \right) &= \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^T L \left(\mathbf{y}(t)^{(n)}, \hat{\mathbf{y}}(t)^{(n)} \right) \\ &= \frac{1}{N} \sum_{n=1}^N \sum_{t=1}^T L \left(\mathbf{y}(t)^{(n)}, g \left(\mathbf{a}(0), \mathbf{x}(1:t)^{(n)}; \mathbf{w} \right) \right) \end{aligned}$$

- We will use Gradient Descent to find parameters that minimize this criterion.

2.2 Backpropagation through time

- Unfolding through time
- Analogy with FCNs

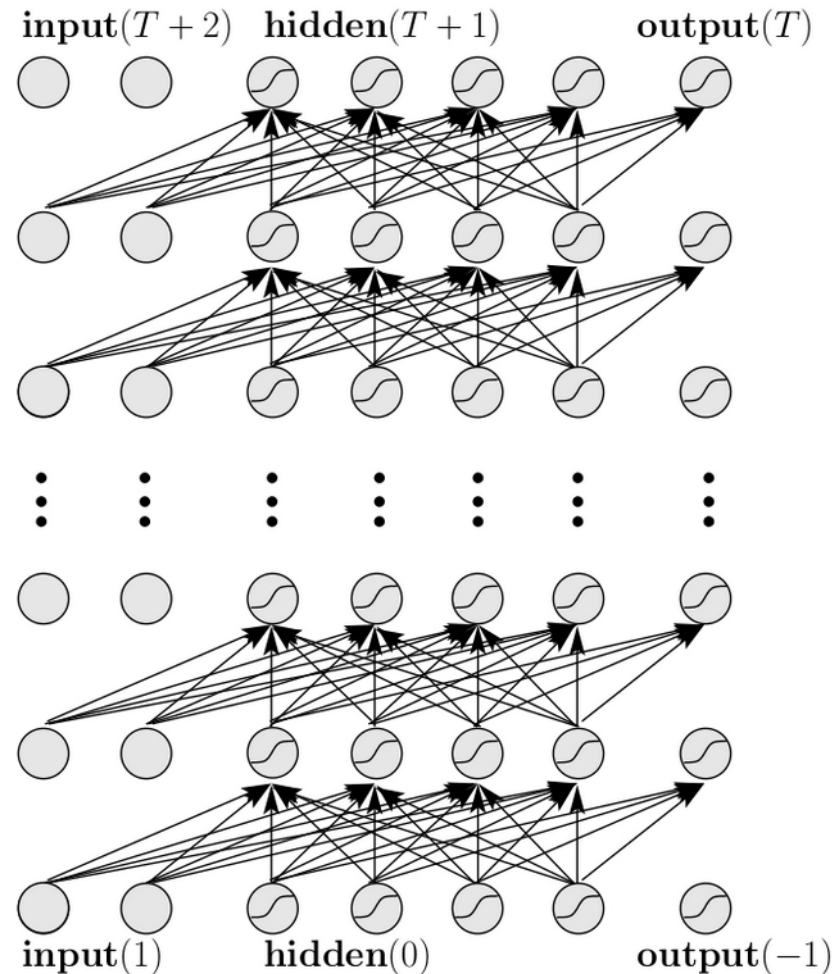


Figure 2.2: The recurrent network from Figure 2.1 left unfolded in time. Dummy inputs at time $(t+1)$ and $(t+2)$ and dummy outputs at $(t-2)$ and $(t-1)$ are used. The input is shifted two time steps against the output to allow the input information to be propagated through the network.

2.2 BPTT

- Delta errors for backpropagation

$$\begin{aligned}\delta(t)^\top &= \frac{\partial L}{\partial \mathbf{s}(t)} = \frac{\partial L}{\partial \mathbf{a}(t)} \frac{\partial \mathbf{a}(t)}{\partial \mathbf{s}(t)} \\&= \left(\frac{\partial L(\mathbf{y}(t), \hat{\mathbf{y}}(t))}{\partial \mathbf{a}(t)} + \frac{\partial L}{\partial \mathbf{s}(t+1)} \frac{\partial \mathbf{s}(t+1)}{\partial \mathbf{a}(t)} \right) \frac{\partial \mathbf{a}(t)}{\partial \mathbf{s}(t)} \\&= \left(\frac{\partial L}{\partial \hat{\mathbf{y}}(t)} \frac{\partial \hat{\mathbf{y}}(t)}{\partial \mathbf{a}(t)} + \frac{\partial L}{\partial \mathbf{s}(t+1)} \frac{\partial \mathbf{s}(t+1)}{\partial \mathbf{a}(t)} \right) \frac{\partial \mathbf{a}(t)}{\partial \mathbf{s}(t)} \\&= \left(\mathbf{e}(t)^\top \text{diag}(\varphi'(\mathbf{V}^\top \mathbf{a}(t))) \mathbf{V}^\top + \delta(t+1)^\top \mathbf{R}^\top \right) \text{diag}(f'(\mathbf{s}(t))).\end{aligned}$$

$$\frac{\partial L}{\partial \mathbf{W}} = \sum_{t=1}^T \delta(t) \mathbf{x}(t)^\top$$

$$\frac{\partial L}{\partial \mathbf{V}} = \sum_{t=1}^T \psi(t) \mathbf{a}(t)^\top$$

$$\frac{\partial L}{\partial \mathbf{R}} = \sum_{t=1}^T \delta(t) \mathbf{a}(t-1)^\top$$

2.5 Vanishing gradient problem for RNNs

- Jacobian through time for preactivations:

$$\mathbf{J}(t) = \frac{\partial \mathbf{s}(t)}{\partial \mathbf{s}(t-1)}$$

- Gradient through multiple time-steps:

$$\frac{\partial s_i(t)}{\partial s_j(t-l)} = \frac{\partial s_i(t)}{\partial \mathbf{s}(t-1)} \frac{\partial \mathbf{s}(t-1)}{\partial \mathbf{s}(t-2)} \dots \frac{\partial \mathbf{s}(t-l+1)}{\partial s_j(t-l)} = \left(\prod_{\tau=0}^{l-1} \mathbf{J}(t-\tau) \right)_{ij}$$

3. Long short-term memory (LSTM)

- RNN architecture that solves the VGP
- Main idea: introduce cell states that memorize and keep Jacobians close to 1

$$c(t) = c(t - 1) + z(t)$$

- Forward pass for LSTM:

$$i(t) = \sigma(\mathbf{W}_i^\top \mathbf{x}(t) + \mathbf{R}_i^\top \mathbf{y}(t - 1))$$

$$o(t) = \sigma(\mathbf{W}_o^\top \mathbf{x}(t) + \mathbf{R}_o^\top \mathbf{y}(t - 1))$$

$$z(t) = g(\mathbf{W}_z^\top \mathbf{x}(t) + \mathbf{R}_z^\top \mathbf{y}(t - 1))$$

$$c(t) = c(t - 1) + i(t) \odot z(t)$$

$$\mathbf{y}(t) = o(t) \odot h(c(t)),$$

3. Long short-term memory LSTM

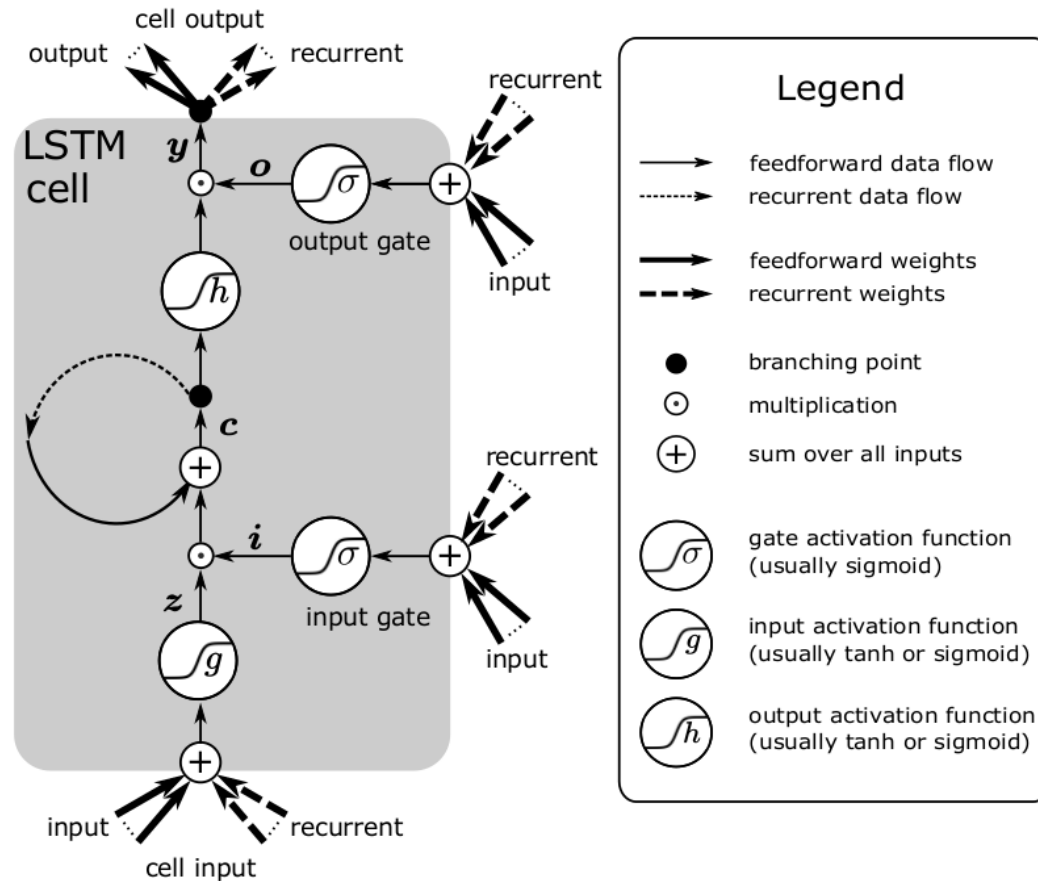


Figure 3.2: The LSTM memory cell. The memory cell input $z(t)$ is often activated by tanh. Subsequently, $z(t)$ is multiplied by an input gate activation $i(t)$. This product is added to the cell state $c(t)$. The memory cell output $y(t)$ is obtained by multiplying the memory cell state activation $h(c(t))$ by an output gate activation $o(t)$. Typically the memory cell activation function h is tanh. The memory cell outputs are then recurrently fed into the cell input and the gates at the next time step $t + 1$ alongside with the new external input $x(t + 1)$. The memory cell state $c(t)$ carries error signals backwards in time without vanishing or exploding gradients.

3. 1 Backpropagation for LSTM

- Most importantly:

$$\frac{\partial \mathbf{c}(t+1)}{\partial \mathbf{c}(t)} \approx \mathbf{I}$$

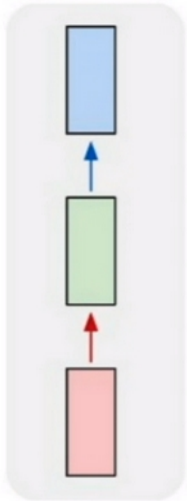
- Rest: see Chapter 3 in Lecture Notes.

3.3 LSTM architecture variants & tricks of the trade

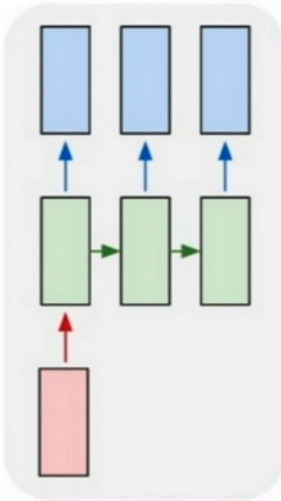
- Many possible variants of LSTM
 - With and without forget gate
 - Lightweight LSTM
 - Ticker steps
 - Gate biases
 - Peephole connections
 - Bi-directional LSTM
 - Convolutional LSTM
 - ...

RNNs: Mapping sequences to sequences

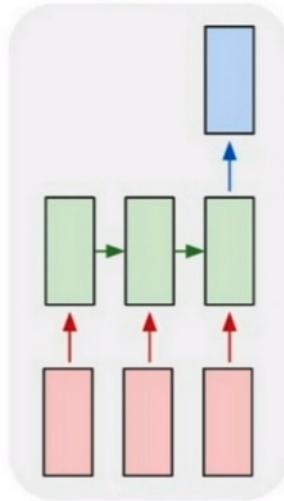
one to one



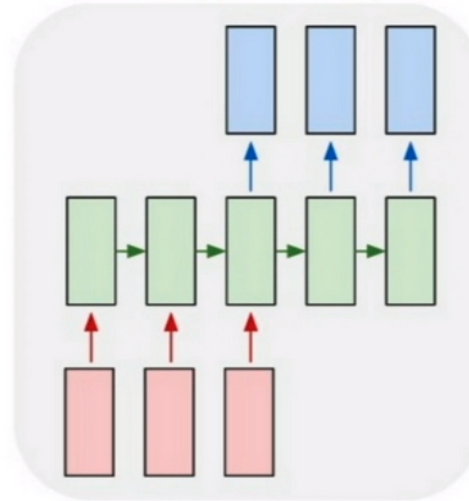
one to many



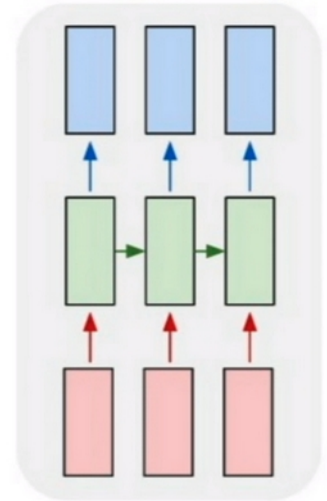
many to one



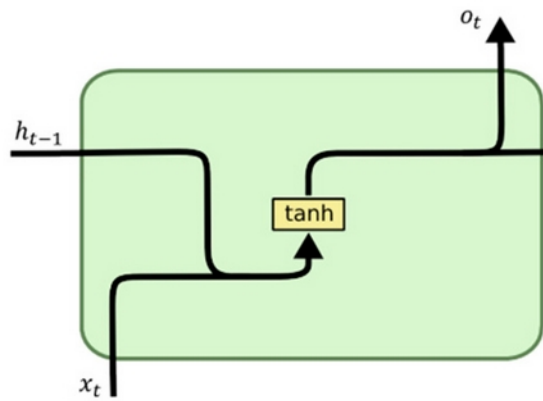
many to many



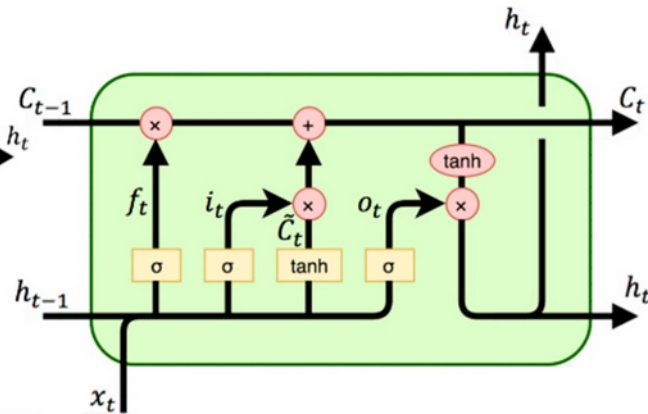
many to many



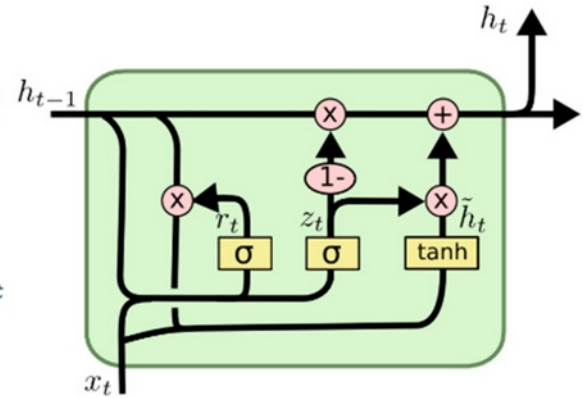
RNN architectures



Simple RNN



LSTM



GRU

Transformer architecture: widely used!

- Uses a set of inputs (rather than a single input);

$$\mathbf{X} = \{x(1), \dots, x(T)\}$$

Note that $\mathbf{X} \in \mathbb{R}^{d \times T}$ is not the training data matrix, but the time series of inputs. Input contains positions encoded as features (position encoding). Otherwise: permutation-equivariant

- The forward pass of a transformer is given by

$$\mathbf{H}^0 = \mathbf{X} \tag{1}$$

$$\mathbf{K}^{(l)} = \mathbf{W}_K^{(l)} \mathbf{H}^{(l-1)} \tag{2}$$

$$\mathbf{Q}^{(l)} = \mathbf{W}_Q^{(l)} \mathbf{H}^{(l-1)} \tag{3}$$

$$\mathbf{V}^{(l)} = \mathbf{W}_V^{(l)} \mathbf{H}^{(l-1)} \tag{4}$$

$$\mathbf{H}^{(l)} = \underbrace{\text{softmax} \left(1/\sqrt{d_k} \mathbf{Q}^{(l)T} \mathbf{K}^{(l)} \right)}_{:=\mathbf{A}^{(l)}} \mathbf{V}^{(l)}, \tag{5}$$

RNN, LSTM and Transformer applications

- Many applications in Natural Language Processing (NLP)
 - Sequence2sequence learning; translation
 - Generating text; “next-character LSTM”
 - Labeling images
 - Answering questions
 - Sentiment detection
 - ...